

Closures

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Lexical Scope Recap

- Scope = the variables a piece of code can "see"
- JS uses **lexical scope**: based on where code is **written**
- Inner functions can read outer variables, never the reverse

```
function outer() {  
  const message = "hello";  
  function inner() {  
    console.log(message); // sees outer's variable  
  }  
  inner();  
}  
outer(); // "hello"
```

What Is a Closure?

- A function **remembers** variables from where it was created
- Survives after the outer function has returned
- Each call captures a **separate** value

```
function makeGreeter(name) {  
  return function () {  
    console.log("Hello, " + name + "!");  
  };  
}  
  
const greetAlice = makeGreeter("Alice");  
greetAlice(); // "Hello, Alice!"
```

The Classic Counter

- Inner function keeps a **private, persistent** variable
- `count` is not global and not on any object

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = makeCounter();  
console.log(counter(), counter(), counter()); // 1 2 3
```

Each Closure Gets Its Own Scope

```
const a = makeCounter();  
const b = makeCounter();  
  
a(); a(); // 1, 2  
b();      // 1 - completely independent of a
```

- Every call to `makeCounter` creates a **new** `count`
- This is why closures work as a factory for isolated state

Private State

- Closures give **true private state**
- Only the exposed functions can change the data

```
function createAccount(start) {  
  let balance = start;  
  return {  
    deposit(n) { balance += n; return balance; },  
    getBalance() { return balance; },  
  };  
}  
  
const acct = createAccount(100);  
console.log(acct.deposit(50)); // 150  
console.log(acct.balance);    // undefined - unreachable
```

The `var`-in-a-loop Pitfall

- `var` has **one shared** binding for the whole loop
- Callbacks all see the final value

```
const funcs = [];  
for (var i = 0; i < 3; i++) {  
  funcs.push(() => i);  
}  
console.log(funcs[0]()); // 3 (not 0!)
```

- The loop finished before any callback ran, and `i` is 3 by then

Fix With `let`

- `let` creates a **new binding per iteration**
- Each closure captures its own value

```
const funcs = [];  
for (let i = 0; i < 3; i++) {  
  funcs.push(() => i);  
}  
console.log(funcs[0](), funcs[1](), funcs[2]()); // 0 1 2
```

- One more reason `let` replaced `var`

Practical Pattern: memoize

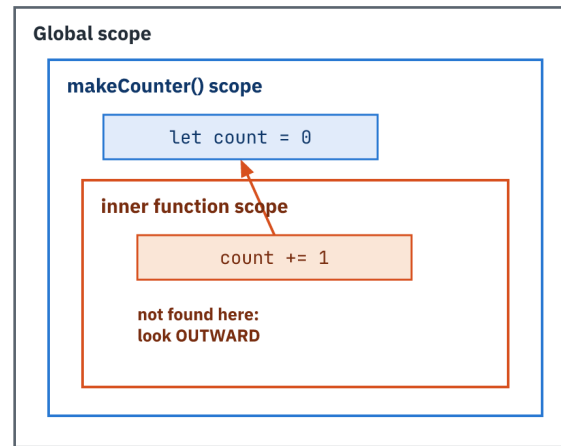
- Private `cache` persists across calls
- Reuses stored answers instead of recomputing

```
function memoize(fn) {  
  const cache = new Map();  
  return function (n) {  
    if (cache.has(n)) return cache.get(n);  
    const result = fn(n);  
    cache.set(n, result);  
    return result;  
  };  
}  
  
const fast = memoize((n) => n * n);  
console.log(fast(4)); // computes → 16  
console.log(fast(4)); // cached   → 16
```

What a Closure Actually Is

Scope Chain and Closures

Lookup goes OUTWARD only. A closure is the link that survives after the outer call returns.



Lookup never goes inward: the outer scope cannot see anything declared inside, and two sibling scopes cannot see each other.

After makeCounter() returns

The makeCounter() frame is gone from the stack, but the returned function still holds its scope link, so count is kept alive and stays private.

```
const a = makeCounter();
const b = makeCounter();
a(); a(); // 1, 2
b(); // 1 - its own count
```

Every call to the outer function makes a NEW scope, so a and b capture two independent counts. That is how closures give you private state without a class.

- The frame is gone from the stack, but the scope link survives
- Every call makes a new scope, so each closure gets its own copy